Stronger Encoders and Structure-Constrained Decoding for DiCo-NLI (Track 1)

Muhammad Muneeb Baig, Syed Kamran Ali Shah, Kashaf Ali Haroon
MS Data Science, SEECS, NUST
CS-878 Deep Learning, Assignment 2: Proposed Approach and Experimental Design

November 2026

Abstract

Assignment 1 (A1) established that a fine-tuned RoBERTa-base reaches 0.765 weighted F1 (W-F1) and 0.739 HardCons on the DiCo-NLI English development set, and that its remaining errors lie mostly on the NEG boundary rather than on direction. We propose a pair-aware system: a fully fine-tuned DeBERTa-v3 cross-encoder that sees both orientations of a pair in the same batch, an optional reversal-consistency loss, NEG class weighting, and inference-time decoding rules that decide both orientations together. We define comparison systems, an ablation plan, the evaluation measures and a complete protocol, and verify an end-to-end implementation. In preliminary three-seed runs, DeBERTa-v3 improves argmax W-F1 from 0.758 to 0.821 and HardCons from 0.759 to 0.839 (both significant under a paired bootstrap). Symmetric decoding adds 2.3 HardCons points at no training cost. A property of the released data (rows with a reversed partner are never NEG) gives the largest gain, to 0.925 W-F1, but it is dataset-specific. The consistency loss shows no significant effect. These are verification results on a development set that informed our design, not final performance claims.

1 Proposed Approach

Task. The input is an ordered pair of short English phrases (t_1, t_2); the output is one of four labels for that ordered pair: EQ (equivalence), FWD (t_1 entails t_2), BWD (t_2 entails t_1) or NEG (opposition, similarity, relatedness or no relation, merged into one class) (DiCo-NLI Organizers, 2026; MacCartney, 2009). The prediction target is one label per row. Many pairs appear in both orientations, and a correct system must then satisfy the reversal rule $f(t_2, t_1) = \mathrm{rev}(f(t_1, t_2))$, with $\mathrm{rev}(\text{FWD}) = \text{BWD}$, $\mathrm{rev}(\text{BWD}) = \text{FWD}$ and $\mathrm{rev}(\text{EQ}) = \text{EQ}$. The expected model behaviour is therefore not only a correct label per row but coherent labels across orientations.

System overview. The system has four parts (Figure 1): (1) *pair-aware preprocessing*, which groups the two orientations of a pair by `pair_id` and tokenizes each orientation as one cross-encoder input; (2) a *fully fine-tuned DeBERTa-v3-base* encoder with a classification head; (3) a *training objective* of class-weighted cross-entropy plus an optional reversal-consistency term; and (4) a *pair-aware decoder* that chooses labels for both orientations together.

Role of the A1 features. A1's most informative engineered features were directional: phrase lengths, word coverage of one phrase by the other, and containment. A cross-encoder reads both phrases jointly with cross-attention, so it can learn these signals from the tokens; we do not feed them explicitly. They remain in the classical comparison system (Section 4), which makes it possible to see what the encoder adds beyond them.

Preprocessing, augmentation and sampling. There is no text normalization: the tokenizers are cased and the phrases are used as released. Each row becomes "[CLS] t_1 [SEP] t_2 [SEP]", truncated at 64 tokens. The longest dev pair has 18 words (mean 5.9), so the limit is not expected to bind. We do not use reversal augmentation: 1,282 of the 1,760 training pairs already appear in both orientations, so reversing them would only duplicate data. Reversing the 478 single NEG rows would also create NEG rows with partners, which never occur in the released data. Instead, batches are sampled by pair: a reversible pair always contributes both orientations to the same batch, and batches are shuffled each epoch. There is no feature selection.

Variation and noise. *Imbalance:* NEG is the smallest class (16% of train) and the hardest; we weight it in the loss and report Macro-F1 alongside W-F1. *Noisy labels:* some gold labels are debatable (A1 found near-paraphrases labelled NEG); we do not model label noise, but an audit is planned (Section 11). *Long inputs:* not an issue at 18 words. *Cross-lingual variation:* this system covers Track 1 only; a multilingual pilot is planned. Domain, speaker and multimodal variation do not apply to this task.

Expected limitations and sources of error. (i) The EQ/FWD/BWD boundary for near-paraphrases, where a small word difference may or may not change entailment; (ii) the heterogeneous NEG class, which merges four relations; (iii) dependence of the structural decoding rules on how the dataset was built; (iv) the small training set (2,738 rows after the validation split).

2 Method Justification

Table 1 links each design decision to evidence from A1. We follow the pattern observation → interpretation → modelling implication.

Encoder. *Observation:* A1's errors concentrate on EQ versus NEG for lexically similar phrases. *Interpretation:* the model must judge whether a small lexical change alters meaning, which depends on word order and relative position. *Implication:* DeBERTa-v3 encodes content and relative position separately (disentangled attention) and is pretrained with replaced-token detection (He et al., 2023). It was also the strongest model in the organisers' pilot (Apaolaza Larraya et al., 2026). We test whether that translates into a better EQ/NEG boundary, rather than assuming it.

Joint decisions over both orientations. *Observation:* RoBERTa almost never swaps FWD and BWD, yet still loses SoftCons and HardCons on pairs where one orientation is called NEG. *Interpretation:* the two orientations are scored independently, so nothing forces their decisions to agree. *Implication:* make the decision at pair level, during training (a consistency regulariser, in the spirit of logic-constrained NLI training (Minervini and Riedel, 2018; Li et al., 2019)) and at inference (symmetric decoding). Consistency benchmarks show that pretrained models are often inconsistent under such transformations (Jang et al., 2022).

Pair structure. *Observation:* the training set has 1,282 pairs in both orientations ($2 \times 1{,}282 = 2{,}564$ rows) and 478 rows in one orientation only; all 478 are NEG, and no paired row is NEG. Dev shows the same pattern (277 pairs, 106 single NEG rows). *Interpretation:* this is a construction artifact of the dataset, similar to the annotation artifacts found in other NLI datasets (Gururangan et al., 2018). Partners can be found from `pair_id` in the participant file, which is available at test time, without any gold label. *Implication:* decoding rules that use it can be applied, but they must be reported separately and described as dataset-specific. It also explains why most top development-leaderboard entries (9 of the other 25 English submissions on 8 November) report SoftCons of exactly 1.0: SoftCons fails a pair whenever either prediction is NEG.

3 System Architecture

Encoder and head. We use `AutoModelForSequenceClassification` with the pretrained `microsoft/deberta-v3-base` weights (about 184M parameters, of which about 98M are the 128k-token embedding matrix). The head takes the first-token representation, applies a dense layer with GELU (the DeBERTa pooler) and a linear layer to four logits. The head is initialised randomly; dropout is the model default (0.1). For RoBERTa the structure is the same, with its own classification head.

Training objective. Let $p_{12} = \mathrm{softmax}(f_\theta(t_1, t_2))$ and $p_{21} = \mathrm{softmax}(f_\theta(t_2, t_1))$. Let π be the label permutation that maps a distribution for (t_2, t_1) onto the label space of (t_1, t_2): it keeps EQ and NEG and swaps FWD and BWD. For each reversible pair in a batch,

$$\mathcal{L}_{\text{cons}} = \tfrac{1}{2}\Big[\mathrm{KL}\big(p_{12} \,\|\, \pi(p_{21})\big) + \mathrm{KL}\big(\pi(p_{21}) \,\|\, p_{12}\big)\Big], \quad (1)$$

and the loss for a batch is

$$\mathcal{L} = \mathrm{CE}_w(y, p) + \lambda\, \overline{\mathcal{L}_{\text{cons}}}, \quad (2)$$

where CE_w is cross-entropy over all rows, with weight w_{NEG} on NEG and 1 elsewhere, and the bar denotes the mean over reversible pairs in the batch. Single rows have no partner, so they receive only CE_w.

Decoding rules. These change only how labels are chosen, so they apply to any trained model:

- **argmax**: $\hat{y} = \arg\max p_{12}$, as in A1.
- **symmetric**: for a row with a partner, $\tilde{p}_{12} = \frac{1}{2}\big(p_{12} + \pi(p_{21})\big)$ and the partner receives $\pi(\tilde{p}_{12})$, so the two labels are consistent by construction.
- **struct_pair**: symmetric, and rows with a partner may not be NEG.
- **struct_full**: struct_pair, and rows without a partner are labelled NEG.

The last two use the pair structure of Section 2. struct_full changes only rows without a partner, which never enter SoftCons or HardCons, so it can raise W-F1 but not HardCons relative to struct_pair.

4 Comparison Systems

All systems are scored on the same official dev set with the same scorer (Table 2).

Simple baselines. The *majority class* predicts FWD for every row. Its accuracy (0.288) equals the FWD share of dev, and its W-F1 (0.129) shows that no single label dominates: FWD, BWD and EQ each cover 26–29% of dev, and NEG 16%. A *length-only* logistic regression tests the heuristic "the longer phrase entails the shorter". It is highly consistent (SoftCons 0.989) but mostly wrong (W-F1 0.464), which is why SoftCons alone cannot be used to compare systems.

Classical baselines. Logistic regression (L2, $C = 1$, L-BFGS, 5,000 iterations, no class weighting) on (a) 26 standardised engineered features (phrase lengths and differences, word coverage and containment in both directions, extra words on each side, Jaccard overlap, identity, negation and number mismatch, and spaCy part-of-speech counts of the non-shared words on each side), (b) TF-IDF unigrams and bigrams with sublinear term frequency, using separate blocks for t_1 and t_2 so that direction is preserved, and (c) both together. These were trained on the full training set in A1.

Pretrained baselines. BERT-base-uncased and RoBERTa-base, fully fine-tuned as cross-encoders with a linear head on the first token and cross-entropy (A1

Evidence from A1 or the data	Design decision	Expected effect
NEG is the weakest class (RoBERTa F1 0.565) and the smallest (16% of train)	NEG class weight $w_{\text{NEG}} = 2$ in the loss; report Macro-F1 and per-class recall	Higher NEG recall; less dominance of the three larger classes
69 near-paraphrases rejected as NEG; EQ F1 only 0.725	Stronger encoder: DeBERTa-v3-base (He et al., 2023)	Better separation of EQ from NEG on lexically similar pairs
FWD/BWD confused only twice in each direction, but 15–19 pairs per seed are NEG in exactly one orientation	Decide both orientations jointly: pair-aware batching, consistency loss, symmetric decoding	Remove one-sided errors; raise HardCons without hurting W-F1
Length-only LR: SoftCons 0.989 but W-F1 0.464	Judge every change on W-F1 and HardCons together, never SoftCons alone	Avoid rewarding systems that are consistent but wrong
In train and dev, every row with a reversed partner is non-NEG and every row without one is NEG	Structure-constrained decoding, reported separately from model gains	Large NEG gain on dev; transfers to test only if the test file is built the same way
Train already holds both orientations of 1,282 pairs	No reversal augmentation; sample whole pairs instead	Consistency loss computable without extra data
DeBERTa-v3 gave NaN losses on Apple MPS in A1	Train in fp32 on CUDA, with gradient clipping	Stable DeBERTa training

Table 1: Evidence from A1, the design decision it motivates, and the expected effect.

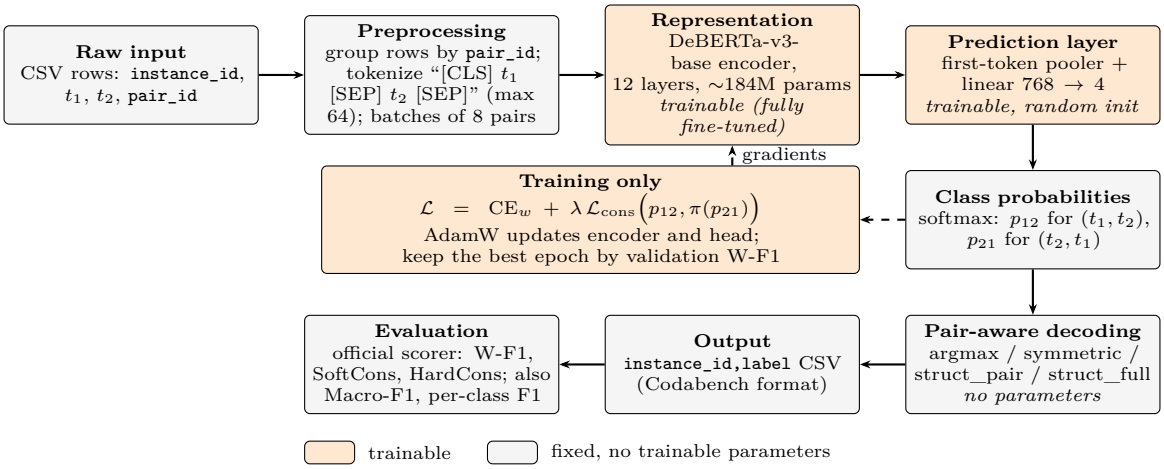


Figure 1: Proposed system. Both orientations of a pair pass through the same encoder and head in the same batch. Nothing is frozen: all encoder and head parameters are updated. π swaps the FWD and BWD probabilities. Single (NEG) rows receive only the cross-entropy term. The decoding rules and the scorer have no trainable parameters.

protocol: AdamW, learning rate 3×10^{-5}, batch 16, 6 epochs, final epoch kept by a rule fixed in advance, Apple M4 GPU). **M0** retrains RoBERTa-base under the A2 protocol, so that the proposed systems can be compared with it on identical splits.

Proposed systems. M1: DeBERTa-v3-base, cross-entropy only. **M2**: M1 + consistency loss ($\lambda = 1$). **M2b**: $\lambda = 0.5$, one seed, as a sensitivity check. **M3**: M2 + NEG weighting ($w_{\text{NEG}} = 2$). Each is evaluated under all four decoding rules, and as a three-seed probability ensemble. What distinguishes them from the baselines is the pair-level treatment of the task, in training (pair-aware batching and the consistency loss) and at inference (pair-aware decoding).

5 Ablation Plan

Each ablation changes one component and answers one question (Table 3). Ablations A–F are run in this assignment as preliminary checks; G–L are planned for A3.

6 Evaluation Measures

6.1 Official metrics

All three official metrics are higher-is-better and range from 0 to 1. For class c with precision P_c, recall R_c

System (argmax)	W-F1	Soft	Hard
Majority class	0.129	0.000	0.000
LR, length only	0.464	0.989	0.628
LR, engineered features	0.593	0.877	0.653
LR, TF-IDF + features	0.620	0.856	0.661
BERT-base (A1)	0.705	0.774	0.693
RoBERTa-base (A1)	0.765	0.804	0.739
M0 RoBERTa, A2 protocol	0.758	0.819	0.759
M1 DeBERTa-v3	0.821	0.892	0.839
M3 + cons. + NEG wt.	0.827	0.878	0.828

Table 2: Comparison systems on dev (official scorer). Soft/Hard: SoftCons/HardCons. A1 rows are from A1; neural rows are three-seed means. Results under the other decoding rules are in Table 6.

and gold support n_c among N rows,

$$\mathrm{F1}_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{W-F1} = \sum_c \frac{n_c}{N}\,\mathrm{F1}_c. \tag{3}$$

W-F1 is the main score. Because it weights classes by support, the three large classes (84% of dev) dominate it, and a weak minority class costs relatively little; we therefore also report Macro-F1.

Let $\mathcal{R}$ be the set of reversible dev pairs ($|\mathcal{R}| = 277$), with gold labels (y_{12}, y_{21}) and predictions $(\hat{y}_{12}, \hat{y}_{21})$. Then

$$\text{SoftCons} = \frac{1}{|\mathcal{R}|}\sum_{\mathcal{R}} \mathbf{1}\big[\hat{y}_{12} \neq \text{NEG} \ \wedge\ \hat{y}_{21} = \text{rev}(\hat{y}_{12})\big], \tag{4}$$

$$\text{HardCons} = \frac{1}{|\mathcal{R}|}\sum_{\mathcal{R}} \mathbf{1}\big[\hat{y}_{12} = y_{12} \ \wedge\ \hat{y}_{21} = y_{21}\big]. \tag{5}$$

SoftCons rewards coherent predictions and ignores gold labels, so it can be high for a wrong system. **HardCons** rewards pairs that are both coherent and correct, so HardCons ≤ SoftCons when gold is coherent. Both are computed only over reversible pairs, which in dev are all non-NEG; single rows affect only W-F1. All three are averaged over items, not over classes.

6.2 Secondary metrics

Accuracy; Macro-F1, which weights the four classes equally and is therefore sensitive to NEG; per-class precision, recall and F1; and the number of strictly one-sided NEG pairs (one orientation NEG, the other not), which isolates the error type that motivated the method.

Uncertainty. We report mean ± sample standard deviation over three seeds. For comparisons we use a paired cluster bootstrap: we resample the 383 dev `pair_id` groups (277 pairs and 106 single rows) with replacement 2,000 times, keeping both orientations of a pair together, and recompute the seed-averaged difference between two systems on each resample. We report the 95% percentile interval and a one-sided bootstrap p value. This measures uncertainty from the finite dev set; seed variance is reported separately.

6.3 Subgroup and robustness analysis

We report accuracy by label, by pair type (rows with and without a partner), by pair length (total words, in thirds) and by lexical overlap (word Jaccard, in thirds). These groups test whether the aggregate score hides weak spots: pairs without a partner are all NEG, long pairs contain more content to compare, and low-overlap pairs cannot be solved by word matching.

7 Experimental Protocol

Table 4 lists the configuration used for every A2 run.

Hyperparameter search. None. The optimiser settings follow A1 and common fine-tuning defaults, and λ and w_{NEG} were fixed before training; the only variation is the single-seed $\lambda = 0.5$ check. This avoids tuning on dev, and leaves a search on the validation split for A3.

Model selection. After each epoch we compute argmax W-F1 on the validation split and keep the best epoch. Dev is never used to select checkpoints or set hyperparameters.

Missing and duplicated instances. No row or label is missing. Seven dev rows repeat a training pair exactly (15 if case is ignored, from 8 `pair_id`s), always with the same label. We keep them, because the official evaluation includes them; they make up 2.3% of dev and slightly favour any trained model.

Class imbalance. NEG weighting in M3, and Macro-F1 and per-class recall in reporting. No resampling.

Leakage prevention. (i) The validation split is grouped by `pair_id`, so a pair and its reverse are never split across training and validation. (ii) Train and dev share no `pair_id`. (iii) The decoding rules find partners only from `pair_id` in the participant file, never from gold labels. (iv) Checkpoints and settings are never chosen on dev. (v) The test set has not been seen.

Local versus server evaluation. Local evaluation runs the organisers' scorer on the public dev labels. We verified that it reproduces the Codabench development-phase score exactly. Hidden-test scores will come only from the server during the official evaluation window (January 2027).

Use of dev. Although dev was not used for selection, it was used to compare decoding rules and configurations, so the final configuration is informed by dev results. Dev scores are therefore not an independent estimate of test performance.

8 Initial Implementation

The system runs end to end in PyTorch and Hugging Face Transformers (Table 5). One Colab notebook drives all experiments and the submission file.

Minimum-requirement checks. *Data load cor-*

ID	Ablation	Hypothesis	Expected interpretation / preliminary outcome
A	RoBERTa (M0) vs DeBERTa-v3 (M1)	A stronger encoder improves the EQ/NEG boundary	Gain isolates the encoder. *Done:* +6.3 W-F1, +7.9 HardCons, both significant
B	Consistency loss on/off (M1 vs M2)	Training-time agreement raises HardCons	Gain isolates the loss. *Done:* no significant effect
C	$\lambda = 1$ vs $\lambda = 0.5$	The loss is not over- or under-weighted	Sensitivity of B to its weight. *Done, one seed:* no improvement
D	NEG weighting on/off (M2 vs M3)	Weighting raises NEG recall without hurting other classes	Inspect NEG precision/recall. *Done:* recall up, W-F1 change not significant
E	Decoding rule: argmax / symmetric / struct_pair / struct_full	Joint decisions remove one-sided errors; structure decides NEG	Separates model gains from inference gains. *Done:* both significant
F	Single seed vs three-seed ensemble	Averaging reduces seed variance	*Done:* +1.1 W-F1, significant
G	Frozen encoder (linear probe) vs fully fine-tuned	Task adaptation is needed beyond generic representations	Accuracy gain against training cost
H	Generic vs NLI-pretrained initialisation (MNLI-style checkpoint)	Prior entailment training helps the EQ/FWD/BWD decision	Gain on paired rows
I	Four-class vs three-class model on paired rows only	When NEG is decided structurally, the model need not learn it	Gain on EQ/FWD/BWD; removes "NEG both ways"
J	Checkpoint selection under argmax vs under the final decoding rule	Selection should match the deployed rule	Different epochs selected; effect on dev
K	Averaged vs joint pair decoding (best of 3 valid labelings)	Exact pair-level decoding beats averaging	Change in HardCons
L	Base vs large encoder; same- vs cross-backbone ensembles	Capacity and diversity help	Gain against cost

Table 3: Ablation plan. Significance is judged with the paired cluster bootstrap of Section 6; preliminary outcomes are detailed in Section 9.

rectly: row counts, label counts and the pair structure match the release. *Inputs become the intended representation:* each batch holds both orientations of its pairs. *The model trains and runs inference:* all 13 runs completed with finite losses. *Predictions are generated:* a probability file per run and split, and decoded label files per rule. *A metric is computed:* the official scorer runs on every file; our own metric code matches it exactly. *The experiment repeats from its configuration:* retraining M3 at seed 42 from its stored configuration reproduced all 660 dev labels of the original run (W-F1 0.829, HardCons 0.820).

Checkpoint saving and loading. With `save_checkpoint=True`, the selected epoch is written with its tokenizer and configuration (about 0.74 GB in fp32). Reloading it from disk in a separate step and predicting dev again gives the same 660 labels and the same official scores as the trained model.

9 Preliminary Results

These runs verify that the pipeline works and show what to expect; they are not final performance claims. Setup: 2,738 training rows, 304 validation rows, all 660 dev rows; 13 training runs (M0–M3 at three seeds, M2b at one); about 40 GPU-minutes in total.

Protocol check. M0 reproduces the A1 RoBERTa result (argmax W-F1 0.758 vs. 0.765, within seed variation), so the new validation split and selection rule do not distort the comparison with A1.

Encoder (ablation A). DeBERTa-v3 is the largest training-side gain: +6.3 W-F1 and +7.9 HardCons under argmax, both with 95% intervals well above zero (Table 7), and with seed standard deviations about four times smaller than RoBERTa's (Table 6). The gain is largest on the two classes A1 identified as hardest, EQ (+0.094 F1) and NEG (+0.104 F1) (Table 8). No NaN losses occurred in fp32.

Decoding (ablation E). Symmetric decoding raises HardCons by 2.3 points for M1 (significant) with W-F1 unchanged, and removes every one-sided NEG pair; it costs no training. The structural rules are the largest gain, +9.8 W-F1 for M1. Table 11 shows where it comes from: every NEG row is now correct and none is lost, because NEG is assigned exactly to the rows without a partner, and the EQ/FWD/BWD rows previously lost to NEG are recovered. This gain reflects the construction of the dataset, not better language understanding. Applied to the saved A1 models, the same rules raise RoBERTa from 0.765 to 0.874 W-F1 and BERT from 0.705 to 0.838, so the effect does not depend on the A2 models.

Consistency loss (ablations B, C): no signifi-

Setting	Value
Data	DiCo-NLI Track 1 (English), organisers' release (GitHub, commit `588968e`, 8 Sep 2026)
Partitions	train 3,042 rows; dev 660 (277 pairs + 106 singles); test hidden until Jan 2027
Validation	10% of train by `pair_id`: 128 pairs + 48 singles (304 rows); split seed 2027, identical for all runs
Tokenization	model tokenizer, cased; pair input; max 64 tokens; dynamic padding
Initialisation	pretrained Hub checkpoints; head random
Optimiser	AdamW, lr 2×10^{-5}, weight decay 0.01
Schedule	linear warm-up (10% of steps), linear decay
Batch	8 pairs (≈16 rows), shuffled per epoch
Regularisation	dropout 0.1 (default); grad. clip 1.0
Epochs / selection	6 epochs; keep the epoch with the best validation argmax W-F1
Loss settings	$\lambda \in \{0, 0.5, 1\}$, $w_{\text{NEG}} \in \{1, 2\}$
Precision	fp32
Seeds	13, 42, 87
Hardware	Google Colab, NVIDIA T4 (16 GB)
Software	Python 3, PyTorch and Hugging Face Transformers (Colab defaults, Nov. 2026), scikit-learn
Cost	≈2 min per RoBERTa run, 3.3 min per DeBERTa run (6 epochs)

Table 4: Training and evaluation configuration, shared by all A2 runs.

Requirement	Implementation
Data loading	`src/data.py`: official CSVs, label map
Preprocessing, representation	`PairCollator`: pairs by `pair_id`, tokenizes both orientations
Model, training	`src/train_a2.py`: `run(**config)`, loss, validation selection
Inference	`predict_probs`; probabilities saved per run
Decoding	`src/decoding.py`: four rules
Evaluation	official scorer; `06_decoding_experiments.py`; `07_a2_analysis.py` (bootstrap, subgroups)
Prediction format	`instance_id,label`; checked against the Codabench rules
Configuration	defaults in `train_a2.py`; every run's full config and history in `runs/<tag>.json`
Checkpoints	`save_checkpoint`, `load_checkpoint`, `predict_from_checkpoint`
Seeds	`set_seed` (Python, NumPy, PyTorch, CUDA)

Table 5: Implementation components.

cant effect. Under argmax, M2 has 1.2 points *lower* HardCons than M1 (interval −2.8 to +0.4), while its one-sided NEG count falls from 9.0 to 6.7 per seed. A likely explanation is that the loss can be satisfied by predicting NEG in both orientations, which removes one-sided errors without making either prediction correct. Under struct_full the difference is +0.4 W-F1 with an interval spanning zero; after two seeds it looked larger, and the third seed reversed it. $\lambda = 0.5$ (one seed) was no better. This is plausible given A1: the loss enforces directional agreement, while the encoder's errors are mostly about NEG, not direction.

Neg weighting (ablation D). Weighting moves the NEG trade-off as intended: recall rises from 0.61 (M1) to 0.71 (M3) and precision falls from 0.64 to 0.59, giving the best NEG F1 and Macro-F1. The W-F1 change over M2 is not significant (+0.7, interval −0.5 to +1.9). We therefore call M3 the best tested configuration, not evidence that each of its components helps.

Ensembling (ablation F). Averaging the three seeds adds 1.1 W-F1 and 1.3 HardCons (both significant) and keeps the order of the systems (Table 10).

Subgroups. The aggregate hides two weak spots (Table 9). Rows without a partner (all NEG) are much harder than paired rows under argmax (0.61 vs. 0.86 for M1); NEG weighting lifts them to 0.71 at a small cost on paired rows. Long pairs are the hardest group, and the encoder helps least there (+2.4 accuracy from M0 to M1, against +6.1 for short pairs). Low-overlap pairs gain most from the stronger encoder (+9.1), while high-overlap pairs, which word matching already handles, gain nothing. Under struct_full the remaining errors are spread across all groups.

Evidence of meaningful predictions. The confusion matrices (Table 11) are strongly diagonal. FWD and BWD are almost never swapped (3–4 rows), so the model has learned direction rather than length or position. Under struct_full, 42 rows (21 pairs) remain wrong, all on the EQ/FWD/BWD decision: 24 EQ rows called entailment and 14 entailment rows called EQ.

External check. We submitted the M3 struct_full ensemble, English only, to the Codabench development phase (submission 1042178). It scored 0.936 W-F1, 1.0 SoftCons and 0.924 HardCons, identical to our local scorer, and ranked 7th of 26 English submissions at the time. This confirms the submission format and the scorer. It is not a test estimate, for the reason given under "Use of dev".

Warnings and failures. (1) DeBERTa-v3 produced NaN losses on Apple MPS in A1; fp32 on CUDA removed the problem. (2) Colab sessions reset between runs; three result files for seed 87 were not copied to Drive and were regenerated from the saved probabilities, with identical scores. (3) A1's "42 pairs with NEG in one direction only" also counted pairs predicted NEG in both directions; the strictly one-sided count is 15–19 per seed. (4) M1's struct_full W-F1 is identical to 16 decimal places for seeds 13 and 42. This is not a duplicated run: the seeds disagree on 24 dev rows,

	argmax		symmetric		struct_pair		struct_full	
System	W-F1	HardCons	W-F1	HardCons	W-F1	HardCons	W-F1	HardCons
M0 RoBERTa	0.758 ±.026	0.759 ±.025	0.768 ±.034	0.821 ±.025	0.803 ±.036	0.869 ±.026	0.889 ±.022	0.869 ±.026
M1 DeBERTa-v3	0.821 ±.006	**0.839** ±.006	0.821 ±.006	**0.862** ±.015	0.853 ±.010	0.904 ±.008	0.919 ±.006	0.904 ±.008
M2 + cons.	0.820 ±.005	0.827 ±.011	0.822 ±.005	0.845 ±.006	0.870 ±.013	0.909 ±.008	0.923 ±.007	0.909 ±.008
M3 + NEG wt.	**0.827** ±.004	0.828 ±.015	**0.824** ±.004	0.842 ±.006	**0.877** ±.010	**0.911** ±.004	**0.925** ±.004	**0.911** ±.004

Table 6: Dev results, mean ± std over seeds 13, 42 and 87 (official scorer). SoftCons is 1.0 for both structural rules by construction; under argmax it is 0.819, 0.892, 0.884 and 0.878 for M0–M3, and under symmetric 0.919, 0.931, 0.910 and 0.906. Bold marks the best mean per column; differences among M1–M3 are not significant (Table 7).

Comparison (seed-averaged)	ΔW-F1 [95% CI]	ΔHardCons [95% CI]
A: M1 vs M0, argmax	+6.3 [+3.0, +9.7]*	+7.9 [+3.7, +11.9]*
B: M2 vs M1, argmax	−0.1 [−1.5, +1.2]	−1.2 [−2.8, +0.4]
B: M2 vs M1, struct_full	+0.4 [−1.0, +1.7]	+0.5 [−1.2, +2.0]
D: M3 vs M2, argmax	+0.7 [−0.5, +1.9]	+0.1 [−1.3, +1.6]
D: M3 vs M2, struct_full	+0.2 [−0.7, +1.2]	+0.2 [−0.8, +1.5]
M3 vs M1, struct_full	+0.5 [−0.7, +1.8]	+0.7 [−0.7, +2.3]
E: M1 symmetric vs argmax	0.0 [−0.6, +0.5]	+2.3 [+1.3, +3.4]*
E: M1 struct_full vs argmax	+9.8 [+7.6, +12.3]*	+6.5 [+4.4, +8.8]*
F: M3 ensemble vs M3 seeds, struct_full	+1.1 [+0.3, +1.9]*	+1.3 [+0.4, +2.3]*

Table 7: Paired cluster bootstrap over dev `pair_id` groups (2,000 resamples), in percentage points. *: 95% interval excludes 0 (one-sided $p < 0.01$). Letters refer to Table 3.

	F1 (argmax)				
System	EQ	FWD	BWD	NEG	Macro-F1
M0	0.719	0.864	0.823	0.516	0.730
M1	0.813	0.882	0.880	0.620	0.799
M2	0.801	0.879	0.883	0.633	0.799
M3	0.809	0.889	0.884	**0.643**	**0.806**

Table 8: Per-class F1 under argmax, mean of three seeds. NEG precision/recall: M0 0.53/0.51, M1 0.64/0.61, M3 0.59/0.71. Under struct_full, NEG F1 is 1.000 by construction.

Group	n	M0 argmax	M1 argmax	M3 argmax	M3 s_full
With partner	554	0.809	0.862	0.845	0.911
Without partner	106	0.506	0.610	0.708	1.000
Short (≤5 words)	312	0.794	0.855	0.862	0.949
Medium (6–7)	210	0.743	0.829	0.837	0.914
Long (>7)	138	0.710	0.734	0.715	0.889
Low overlap (J≤.25)	250	0.688	0.779	0.773	0.909
Mid overlap	299	0.778	0.843	0.846	0.938
High overlap (J>.5)	111	0.874	0.859	0.874	0.928

Table 9: Dev accuracy by subgroup, mean of three seeds. Length is total words in the pair; J is word Jaccard overlap.

but each gets exactly 12 of them right, in mirrored FWD/BWD-versus-EQ swaps, so the confusion counts coincide.

Issues to resolve before the full experiments. Confirm that the test file keeps the pair structure; align checkpoint selection with the decoding rule; redesign the consistency objective so that it cannot be satisfied by NEG in both orientations; and run more seeds.

	argmax		struct_full	
Ensemble	W-F1	HardCons	W-F1	HardCons
M0	0.784	0.794	0.912	0.895
M1	0.823	0.845	0.928	0.913
M2	0.830	0.845	0.930	0.917
M3	**0.832**	0.830	**0.936**	**0.924**

Table 10: Three-seed probability ensembles on dev (single systems, not means). The M3 struct_full ensemble was submitted to the Codabench development phase.

	argmax				struct_full			
Gold	EQ	FWD	BWD	NEG	EQ	FWD	BWD	NEG
EQ	**137**	7	6	24	**150**	12	12	0
FWD	7	**165**	1	17	7	**181**	2	0
BWD	8	2	**166**	14	7	2	**181**	0
NEG	13	7	8	**78**	0	0	0	**106**

Table 11: Confusion matrices of the M3 ensemble on dev (rows: gold, with 174 EQ, 190 FWD, 190 BWD, 106 NEG; columns: predicted).

10 Reproducibility and Limitations

Reproducibility. The repository holds the exact code used for every reported number, the per-run configuration and training history (`runs/<tag>.json`), the dev and validation probabilities of every run, the decoded

label files, and a manifest that maps each number to its configuration and files. All scores were recomputed from the saved probabilities with the official scorer, matching the original runs to within 10^{-16}. The data split, preprocessing and scoring scripts were the same for all systems. Seeds are fixed, and retraining M3 at seed 42 reproduced every dev label of the original run (Section 8). GPU kernels are not guaranteed to be deterministic, so we still report seed means and standard deviations rather than single runs.

Limitations. (1) The structural rules depend on a property of train and dev that may not hold for the test set; we will report test results with and without them. (2) Dev informed the choice of configuration, so dev scores are optimistic. (3) Three seeds are few; the bootstrap intervals cover dev-sampling uncertainty but not every source of variation. (4) Checkpoints are selected by argmax validation W-F1, although the best results use structured decoding. (5) No hyperparameter search was run. (6) Only English is covered. (7) Some gold labels are debatable, and we have not yet measured how many.

11 Planned Experiments

Under the structural rules every remaining error is an EQ/FWD/BWD decision on a paired row (21 of 277 pairs for the best ensemble). Assignment 3 will target that decision and complete the ablations G–L of Table 3:

- **Frozen vs fine-tuned encoder** (G), to measure how much task adaptation adds.
- **NLI-pretrained initialisation** (H), from a DeBERTa-v3 checkpoint already fine-tuned on MNLI-style data.
- **Three-class paired model** (I), trained only on paired rows, which also removes the "NEG both ways" solution that undermined the consistency loss.
- **Checkpoint selection under the final rule** (J) and **joint pair decoding** (K), choosing among the three valid labelings (EQ,EQ), (FWD,BWD), (BWD,FWD) by summed log-probability.
- **DeBERTa-v3-large and cross-backbone ensembles** (L) (Lakshminarayanan et al., 2017).
- **More seeds and a hyperparameter search on the validation split**, with the same bootstrap tests.
- **Error analysis** of the remaining wrong pairs, including a gold-label audit.
- **Multilingual tracks**: the pilot code (mDeBERTa-v3, in-language and zero-shot settings for Spanish, Basque and mixed pairs) is in the repository and will be run in A3.
- **Test submission** in the January 2027 evaluation window, with and without the structural rules.

References

Jon Felix Apaolaza Larraya, Begoña Altuna, Aitor Soroa, and Inigo Lopez-Gazpio. Assessing logical coherence of LLMs via fine-grained NLI. In *Proceedings of the Fifteenth Language Resources and Evaluation Conference (LREC 2026)*, pages 5431–5444, Palma, Mallorca, Spain, 2026. European Language Resources Association. doi: 10.63317/4prei82n6ev9.

DiCo-NLI Organizers. SemEval 2027 task 2: Directional-consistent fine-grained natural language inference (DiCo-NLI). `https://github.com/ilopezgazpio/SemEval-2027-Task-2-DiCo-NLI`, 2026. Accessed 4 October 2026.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2*, pages 107–112. Association for Computational Linguistics, 2018. doi: 10.18653/v1/N18-2017.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *The Eleventh International Conference on Learning Representations*, 2023.

Myeongjun Jang, Deuk Sin Kwon, and Thomas Lukasiewicz. BECEL: Benchmark for consistency evaluation of language models. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 3680–3696, Gyeongju, Republic of Korea, 2022. International Committee on Computational Linguistics. URL `https://aclanthology.org/2022.coling-1.324/`.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Tao Li, Vivek Gupta, Maitrey Mehta, and Vivek Srikumar. A logic-driven framework for consistency of neural models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3924–3935, 2019.

Bill MacCartney. *Natural Language Inference*. PhD thesis, Stanford University, 2009.

Pasquale Minervini and Sebastian Riedel. Adversarially regularising neural NLI models to integrate logical background knowledge. In *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pages 65–74, 2018.